

ArchiFlow AI – Developer Task Breakdown (For AI Agent Coder)

1. Purpose of This Document

This document translates the **Cost-Safe Architecture & Pricing Guardrails** into a **clear, executable task breakdown** for an AI agent coder (and human developers).

The goal is to ensure: - Correct system behavior - Cost control by design - No accidental bypass of quotas, caching, or async execution

This document should be treated as a **build checklist**.

2. Development Principles (Must Be Followed)

1. No direct AI calls from frontend or API routes
 2. All AI work must be async and quota-checked
 3. Every expensive action must be measurable and capped
 4. Fail fast before cost is incurred
 5. Prefer simplicity over cleverness
-

3. Frontend Tasks (Web Application)

3.1 Core UI

- Implement diagram editor UI (nodes, edges, layers)
- Implement file upload UI with size validation (max 10MB)
- Implement persona selector (Business / Developer / Security / Trainer)

3.2 Client-Side Restrictions

- Disable submit button if limits are exceeded
- Display remaining quotas clearly
- Prevent multiple concurrent job submissions

3.3 API Interaction Rules

- Frontend must only call backend APIs
- Frontend must never:
 - Call AI providers directly
 - Perform document parsing
 - Execute long-running logic

4. API Gateway Tasks (Cost Gatekeeper)

4.1 Authentication & Authorization

- Implement user authentication (email/OAuth)
- Validate active subscription

4.2 Quota Enforcement

- Check remaining monthly quotas for:
 - Diagrams
 - Uploads
 - Revisions
 - Exports
- Reject requests if any quota is exceeded

4.3 Request Validation

- Validate file size
- Validate diagram complexity (node & layer count)
- Enforce concurrency limit (1 active job per user)

4.4 Rate Limiting

- Apply per-user and per-IP rate limits
-

5. Core Backend Tasks (Cloud Run)

5.1 Data Models

- User
- SubscriptionPlan
- UsageCounter (monthly reset)
- DiagramMetadata
- Job
- CacheEntry

5.2 Business Logic

- Increment usage counters only after job acceptance
- Lookup cache before enqueueing AI jobs
- Store job states (pending, running, completed, failed)

5.3 Export Handling

- Generate exports only via async jobs

- Store export files in Cloud Storage
-

6. Async Job Processing Tasks

6.1 Job Types

- Document parsing
- Diagram generation
- Diagram revision
- Diagram export

6.2 Execution Rules

- Max execution time per job
- Max retries: 2
- Automatic job termination on timeout

6.3 Error Handling

- Log errors with cost impact tag
 - Do not auto-retry jobs that exceed cost ceiling
-

7. AI Orchestration Layer Tasks

7.1 Provider Selection

- Abstract AI provider interface
- Support configurable provider routing (ChatGPT, Copilot, Gemini)

7.2 Cache Logic

- Generate deterministic cache keys
- Return cached result when available
- Persist cache results

7.3 Cost Guardrails

- Enforce per-job cost ceiling
 - Reject jobs that exceed allowed complexity
-

8. Storage & Data Tasks

8.1 Cloud Storage

- Store uploaded documents
- Store generated diagrams
- Store exports

8.2 Database

- Persist user plans
- Persist usage counters
- Persist job metadata

8.3 Cache Store (Optional)

- Implement Redis/Memorystore for hot cache
-

9. Monitoring & Safety Tasks

9.1 Logging

- Log every AI invocation
- Log quota rejections
- Log cache hits vs misses

9.2 Budget Protection

- Integrate Google Cloud budget alerts
- Implement throttling mode trigger

9.3 Kill Switch

- Implement global AI-disable flag
 - Allow cache-only operation mode
-

10. Testing Requirements

10.1 Mandatory Tests

- Quota exhaustion scenarios
- Concurrent job rejection
- Cache hit behavior
- Job timeout handling

10.2 Cost Simulation

- Simulate worst-case user behavior
 - Verify monthly cost does not exceed allocation
-

11. Explicit Do-Not-Implement List

The following must not be implemented in Phase 1: - Unlimited usage - Live AI editing - Background AI analysis loops - Heavy OCR pipelines

12. Completion Criteria

The system is considered complete when: - All quotas are enforced in code - No AI call bypasses orchestration layer - All expensive tasks are async - Budget guardrails are active

Any deviation requires product-owner approval.